Parcial Algoritmos y Programación II 7541 Curso Calvo (2º instancia) 7 de Diciembre 2009

Padrón: **Apellido y Nombre:**

Mail:

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida

```
int main(){
  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;
  H = 68;
  G = 'A';
  A = new int;
  (*A) = H;
  F = &H;
  cout << (*F) << (*A) << endl;;
  B = &F;
  (**B) = 65;
  cout << (*F) << (*A) << (**B) << endl;

  F = new int[3];
  (**B) = 70;

  for (int i = 0; i < 3; i++) {
    F[i] = (*A) + i;
  }
  cout << (*F) << (**B) << endl;

  D = (char*)A;
  E = (char*)(F+1);
  C = (*B);
  C[1] = H;

  cout << (*D) << (*E) << (*C) << endl;
  (*D) = 'B';
  (*E) = 'B';

  for(int i = 0; i < 3; i++) {
    F[i] = 65;
  }
  C = C + 2;
  cout << (*D) << (*E) << (*C) << endl;

  delete A;
  delete[] F;
  return 0;
}
```

2) Implementar para la clase ListaDoblementeEnlazada del método

```
/* pre : la lista tiene al menos tantos elementos como posicionElemento.
 * post: remueve el elemento localizado en la posición posicionElemento de la lista.
 */
void remover(unsigned int posicionElemento);
```

3) Implementar la clase BuscadorDeNoticias a partir de las siguientes especificaciones:

```
class BuscadorDeNoticias {
  public:
    /* post: busca en noticias aquellas Noticias que tengan asociada al menos una de las categorias dadas en conCategorias
     *       y que NO contengan ninguna de las categorias de sinCategorias.
     *       Devuelve una nueva Lista con todas las Noticias encontradas. */
    Lista<Noticia*>* buscarNoticias(Lista<Noticias*>* noticias,
                                    Lista<Categoria*>* conCategorias, Lista<Categoria*>* sinCategorias);
};
```

```
class Noticia {

  public:
    /* post: inicializa la noticia con el título y cuerpo
     *       indicados, sin categorias asociadas.
     */
    Noticia(string titulo, string cuerpo);
    /* post: devuelve el título de la noticia.
     */
    string getTitulo();
    /* post: devuelve el cuerpo de la noticia.
     */
    string getCuerpo();
    /* post: devuelve las categorias asociadas a la noticia.
     */
    Lista<Categoria*>* getCategorias();
};
```

```
class Categoria {

  public:
    /* post: inicializa la categoria con el nombre y la
     *       descripción indicadas.
     */
    Categoria(string nombre, string descripcion);
    /* post: devuelve el nombre que la identifica.
     */
    string getNombre();
    /* post: devuelve la descripción de la categoria.
     */
    string getDescripcion();
};
```

4) Diseñar la especificación e implementar los TDAs que modelen un **Tren**, cumpliendo las siguientes características:
El **Tren** se compone de una serie de **Vagones**. Cada **Vagón** posee una capacidad de carga (medida en toneladas), una capacidad de pasajeros y una longitud (medida en metros). Se deben poder agregar y retirar **Vagones** del **Tren**. Proveer métodos para conocer la longitud total, capacidad total de pasajeros y capacidad total de carga del **Tren**.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Mantener apagados celulares, i-pod, reproductores de mp3, etc. durante el examen
Duración del examen : 2 horas y 30 minutos